Автономная некоммерческая организация профессионального образования
«Международная Академия Информационных Технологий «ИТ ХАБ» (АНО ПО «ИТ ХАБ»)
115093, г. Москва, вн. тер. г. муниципальный округ Даниловский, ул. Дубининская, д. 96
ИНН 9709063913 КПП 772501001
+7 (499) 350-66-04
Ithub.ru

Специальность
10.02.05
(шифр)

дневное
(отделение)

ПРАКТИЧЕСКАЯ РАБОТА

Автоматизированная сборка решений (make)
(тема практической работы)

Выполнил студент группы 4ИБ1-.23

К.А. Мызочкин

Москва, 2026

Цель работы

Освоить автоматизированную сборку программы C++ средствами make, получение отладочной версии и очистку результатов сборки. Перенести алгоритм проверки шахматных угроз на Java, собрать проект с помощью Gradle и проверить корректность работы модульными тестами JUnit.

Исходная задача

На шахматной доске 8 × 8 заданы координаты белого короля, черной ладьи и черного слона. Программа определяет, угрожают ли фигуры королю с учетом возможного перекрытия линий атаки другой черной фигурой. Координаты вводятся через аргументы командной строки.

1. Автоматизация сборки C++

В папке cpp находится Makefile. Для выбора компилятора используется переменная CXX = g++. Переменные CXXFLAGS, SOURCES, TARGET и DEBUG_TARGET задают флаги компиляции, исходные файлы и пути результатов сборки. Цели all, debug, clean, run и help позволяют управлять сборкой. При необходимости пересборку можно принудительно выполнить командой make -B.

```
cd cpp
make
make debug
ls -l build
./build/a.out 4 4 4 8 8 8
file build/a_debug.out
make clean
ls -l build
```

По результатам выполнения в GitHub Codespaces команда make создала build/a.out размером 18 400 байт, а make debug создала build/a_debug.out размером 96 856 байт. Команда file подтвердила наличие отладочной информации (with debug_info, not stripped). После make clean каталог build был удален.

```
ChessMath: version 1
Rook attacks: 1
Bishop attacks: 1
King is under attack: 1
```

2. Перенос алгоритма на Java

Алгоритм реализован в классе ChessMath; класс Main принимает шесть аргументов командной строки и выводит результат. Для автоматизированной сборки применяется Gradle, для модульного тестирования используется JUnit Jupiter.

В проекте используется конфигурация build.gradle с плагинами application и java. Для работы тестов с Gradle 9.7.1 к зависимостям добавлена библиотека org.junit.platform:junit-platform-launcher.

```
cd java
gradle --version
gradle tasks
gradle clean build
gradle run --args="4 4 4 8 8 8"
```

Команда gradle tasks выводит доступные задачи, включая build, test, run, clean и jar. Задача build компилирует исходники, выполняет проверки и формирует результаты сборки. Задача test запускает JUnit. Задача run выполняет Java-программу. Задача clean удаляет каталог build. Задача jar создает архив классов.

3. Проверочные результаты

В GitHub Codespaces использовались OpenJDK 25.0.4.1 и Gradle 9.7.1. Команда gradle clean build завершилась сообщением BUILD SUCCESSFUL in 3s. Все семь тестов ChessMathTest прошли успешно, ошибок и пропущенных тестов нет.

Тест	Результат
rookBlockedByBishop	PASSED
bishopBlockedByRook	PASSED
noPieceAttacks	PASSED
rejectsSameSquare	PASSED
bothPiecesAttack	PASSED
onlyBishopAttacks	PASSED
rejectsOutOfBounds	PASSED

Проверочный запуск Java-программы:

```
gradle run --args="4 4 4 8 8 8"
Rook attacks: true
Bishop attacks: true
King is under attack: true
BUILD SUCCESSFUL in 1s
```

4. Файлы сборки и отчет тестирования

После сборки скомпилированные классы Java находятся в java/build/classes/java/main. JAR-файл формируется задачей gradle jar в каталоге java/build/libs. Его можно запускать через java -cp <имя_файла.jar> ru.ithub.chess.Main 4 4 4 8 8 8. Основная проверка запуска в данной работе выполнена командой gradle run.

HTML-отчет тестирования расположен по пути java/build/reports/tests/test/index.html. Для корректного просмотра вместе с index.html сохраняются связанные файлы CSS, JavaScript и страницы отдельных тестов. Полный отчет приложен в архиве junit-report.zip.

Заключение

Цели Makefile проверены: обычная сборка, отладочная сборка и очистка работают. Алгоритм перенесен на Java, Gradle собирает проект, семь модульных тестов JUnit завершились успешно. Результаты подтверждены выводом терминала и HTML-отчетом.

Репозиторий и материалы

GitHub: https://github.com/bancik222/KT4

Материалы: исходные файлы C++ и Java, Makefile, build.gradle, архив junit-report.zip, снимки терминала с результатами сборки и тестирования.